



项目骨架与代码规范



代码层面的执行规范:仓库结构 / SQL 模板 / RLS 模板 / pgTAP 模板 / TS 规范 / commit 规范。所有模板可直接复制使用。

§1 仓库目录结构

```
├─ apps/  
│   └─ web/                                # Next.js 应用  
│       └─ app/  
│           ├── (public)/                  # SSR + ISR  
│           │   ├── u/[username]/  
│           │   └─ w/[work_id]/[slug]/  
│           ├── (private)/                # 登录后路由  
│           │   ├── library/  
│           │   ├── dashboard/  
│           │   ├── add/  
│           │   └─ settings/  
│           ├── api/                      # webhook / cron  
│           └─ layout.tsx  
│       ├── components/  
│       │   ├── ui/                      # shadcn 原子组件  
│       │   └─ domain/                  # 业务组件  
│       └─ lib/  
│           ├── supabase/                # client / server / admin  
│           ├── registry/                # Media Type Registry  
│           ├── providers/               # Provider Adapters  
│           ├── events/                  # 事件真源三层封装  
│           ├── mspf/                    # 导入导出  
│           ├── rate-limit/              # Upstash  
│           └─ result.ts                 # Result<T, E>
```

```

|       |─ actions/           # Server Actions
|       |─ schemas/          # zod schemas
|─ supabase/
|   |─ migrations/           # YYYYMMDDHHMM_xxx.sql
|   |─ seed.sql
|   |─ functions/            # Edge Functions(reconcil
er)
|─ test/
|   |─ rls/                  # pgTAP
|   |─ e2e/                  # Playwright(P1+)
|─ docs/
|   |─ adr/                  # 独立 ADR 草案
|   |─ runbooks/             # DR 演练 / 故障处理

```

P0 暂不拆 monorepo,直接 `apps/web/` 即可。

§2 包管理 / Node

```

{
  "engines": {
    "node": ">=20.0.0",
    "pnpm": ">=9.0.0"
  },
  "packageManager": "pnpm@9.x.x"
}

```

§3 SQL Migration 规范

3.1 文件命名

```

supabase/migrations/202605161430_create_profiles.sql
└─ YYYYMMDDHHMM_短描述.sql

```

一次 migration 一个目的。

3.2 audit 字段标准集(业务表必备)

```

created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
updated_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
deleted_at      TIMESTAMPTZ,
created_by      UUID REFERENCES profiles(id),
updated_by      UUID REFERENCES profiles(id),
deleted_by      UUID REFERENCES profiles(id),
delete_reason    TEXT

```

`updated_at` trigger:

```

CREATE TRIGGER set_updated_at
BEFORE UPDATE ON <table>
FOR EACH ROW
EXECUTE FUNCTION moddatetime(updated_at);

```

3.3 新表 checklist

- ☐ audit 字段
- ☐ `id UUID PRIMARY KEY DEFAULT gen_random_uuid()`
- ☐ RLS `ENABLE` + 至少一条 policy
- ☐ pgTAP 测试(`test/rls/<table>.sql`)
- ☐ 含 PII → 加入 `docs/runbooks/account-deletion-traversal.md`

3.4 软删除

不写 `DELETE` 业务路径,统一 `UPDATE SET deleted_at = now()`。RLS 自动过滤 `deleted_at IS NULL`。

§4 RLS Policy 模板

4.1 owner-only(私密笔记 / 草稿等)

```

ALTER TABLE user_private_notes ENABLE ROW LEVEL SECURITY;

CREATE POLICY "owner_select"
ON user_private_notes FOR SELECT
USING (profile_id = auth.uid() AND deleted_at IS NULL);

```

```
CREATE POLICY "owner_modify"
ON user_private_notes FOR ALL
USING (profile_id = auth.uid())
WITH CHECK (profile_id = auth.uid());
```

4.2 owner-write / public-conditional-read

```
CREATE POLICY "owner_write"
ON user_entries FOR ALL
USING (profile_id = auth.uid())
WITH CHECK (profile_id = auth.uid());

CREATE POLICY "public_read_visible"
ON user_entries FOR SELECT
USING (
    visibility_scope_snapshot = 'public'
    AND deleted_at IS NULL
);
```

4.3 字段级 visibility(推荐 view + RLS)

```
CREATE VIEW public_entries_v AS
SELECT
    id,
    profile_id,
    work_id,
    edition_id,
    CASE WHEN (field_visibility_json->>'rating') = 'public' T
HEN rating_x10 END AS rating_x10,
    CASE WHEN (field_visibility_json->>'review') = 'public' T
HEN review END AS review
FROM user_entries
WHERE visibility_scope_snapshot = 'public'
    AND deleted_at IS NULL;
```

4.4 RLS 分类速查

P0 所有表归类到三种 RLS 模式之一,对应 §4.1–§4.3 模板:

模式	模板	适用表
owner-only	§4.1	<code>entries</code> (写)、 <code>progress_log</code> 、 <code>private_notes</code> 、 <code>tags</code> 、 <code>entry_tags</code>
shared-public-read	§4.2 变体	<code>works</code> 、 <code>editions</code> 、 <code>external_ids</code> (写限 INSERT, 不允 UPDATE/DELETE)
字段级 visibility view	§4.3	<code>profiles</code> → <code>public_profile_view</code> ; <code>entries</code> → <code>public_entry_view</code>

每张表的完整读/写矩阵见  [DB Schema v1.0.0 设计 §7](#)。不允许新增第四种 RLS 模式;如需走 ADR 修订。

§5 pgTAP 测试模板

`test/rls/<table>.sql` 骨架:

```
BEGIN;
SELECT plan(12);

-- 准备三个身份
SELECT _set_auth_user('00000000-0000-0000-0000-000000000000
1'::uuid); -- owner
INSERT INTO user_entries (id, profile_id, ...) VALUES
('...', '00000000-...', ...);

-- 1. owner 可读自己
SELECT ok(EXISTS(SELECT 1 FROM user_entries WHERE id =
'...'),
'owner can SELECT own entry');

-- 2. owner 可写自己
SELECT lives_ok($$ UPDATE user_entries SET rating_x10 = 40
WHERE id = '...' $$,
'owner can UPDATE own entry');

-- 3. owner 不能伪造 profile_id
SELECT throws_ok($$ INSERT INTO user_entries (profile_id,
...) VALUES ('other-uuid', ...) $$,
```

```

'42501',
'owner cannot INSERT as other user');

-- 切换 other
SELECT _set_auth_user('00000000-0000-0000-0000-000000000000
2'::uuid);

-- 4. other 不可读 private
SELECT is((SELECT count(*) FROM user_entries WHERE id =
'...'), 0)::bigint,
'other user cannot SELECT private entry');

-- 切换 anonymous
SELECT _set_auth_user(NULL);

-- 5. anonymous 不可读 private
-- 6. anonymous 可读 public
-- ...

SELECT * FROM finish();
ROLLBACK;

```

`_set_auth_user` 辅助函数放 `test/rls/_helpers.sql`, 用 `SELECT set_config('request.jwt.claims', ...)` 实现。

§6 TypeScript 规范

6.1 Server Action 模板

```

// actions/entries/create-entry.ts
"use server"
import { z } from "zod"
import { createClient } from "@lib/supabase/server"
import { ok, err, type Result } from "@lib/result"

const CreateEntryInput = z.object({
  editionId: z.string().uuid(),
  status: z.enum(["want_to_read", "reading", "finished", "a

```

```

    bandoned" ]),
  })

export type CreateEntryError =
  | { kind: "validation"; message: string }
  | { kind: "not_authenticated" }
  | { kind: "edition_not_found" }
  | { kind: "internal"; cause: unknown }

export async function createEntry(
  input: unknown,
): Promise<Result<{ entryId: string }, CreateEntryError>> {
  const parsed = CreateEntryInput.safeParse(input)
  if (!parsed.success) {
    return err({ kind: "validation", message: parsed.error.
message })
  }
  const supabase = createClient()
  const { data: { user } } = await supabase.auth.getUser()
  if (!user) return err({ kind: "not_authenticated" })
  // ... 业务逻辑
  return ok({ entryId: "..." })
}

```

6.2 Result<T, E>

```

// lib/result.ts
export type Result<T, E> =
  | { ok: true; value: T }
  | { ok: false; error: E }

export const ok = <T>(value: T): Result<T, never> => ({ ok:
true, value })
export const err = <E>(error: E): Result<never, E> => ({ o
k: false, error })

```

不 throw 给前端。前端按 `if (result.ok)` 分支处理。

6.2a 全局 ErrorCode 目录与 Error shape

每个 Server Action 可自定义 discriminated union 错误(如 §6.1),但 `kind` 字段值必须取自全局 **ErrorCode** 目录,否则前端 toast / i18n 会脱控:

```
// lib/errors.ts
export const ErrorCode = {
  VALIDATION: "validation",
  NOT_AUTHENTICATED: "not_authenticated",
  NOT_AUTHORIZED: "not_authorized",
  RATE_LIMITED: "rate_limited",
  NOT_FOUND: "not_found",
  CONFLICT: "conflict",
  PROVIDER_FAILED: "provider_failed",
  INTERNAL: "internal",
} as const
export type ErrorCode = typeof ErrorCode[keyof typeof ErrorCode]

export type StandardError = {
  kind: ErrorCode
  message?: string // dev-only,不直接展示给用户
  field?: string // 表单字段定位
  details?: Record<string, unknown>
}
```

前端按 `error.kind` 走 i18n key:

```
const message = t(`error.${result.error.kind}`)
```

红线:

- 不要让 `error.message` 直接渲染到 UI(可能含数据库错误细节)
- 自定义 domain error(如 `edition_not_found`)要么映射回 `NOT_FOUND`,要么在 i18n bundle 新增 key `error.edition_not_found`
- 每新增 ErrorCode 必须同步更新 zh-CN / en-US i18n bundle 两个键(见 [🌐 i18n key bundle 雏形](#))

6.3 Supabase 客户端三套

lib/supabase/client.ts	浏览器, anon key
lib/supabase/server.ts	Server Component / Action, 带 session
lib/supabase/admin.ts	Service Role, 仅 migration / cron / webhook

绝不在客户端 import admin。

6.4 zod schema 放置

与 server action 同名对应:

```
schemas/entries/create-entry.ts
actions/entries/create-entry.ts
```

§7 Commit / Branch / PR

7.1 Commit 格式(Conventional Commits)

<type>(<scope>): <短描述>

[正文]

[footer]

type: feat / fix / chore / refactor / docs / test / migration / adr

scope: entries / progress / rls / mspf / 等模块名

例:

```
feat(entries): add createEntry server action with zod validation
migration(progress): partition progress_logs by month
adr(032): draft soft-dedup proposal
```

7.2 Branch

```
main                # 受保护, PR merge
feat/<task-id>-<slug> # 对应任务编号
fix/<short-slug>
migration/<short-slug>
```

7.3 PR 描述模板

```
## 任务
关联 Task NN / ADR-NNN

## 变更摘要
- ...

## 不变量自检(协作章程 §3)
- [ ] 未引入媒介硬编码
- [ ] 未让公开查询 JOIN private_notes
- [ ] 未跳过 progress_logs 直写 snapshot
- [ ] (改 schema) 已更新 audit / RLS / pgTAP / 注销遍历清单

## 验收
对照任务卡的 验收 逐项 
```

§8 CI 流水线要点

`.github/workflows/` 至少包含:

- `lint-and-type.yml`: eslint + tsc
- `migration-smoke.yml`: 空 PG 跑全部 migration + schema diff(ADR-044)
- `pgtap.yml`: apply migrations + 跑 pgTAP(ADR-018)
- `feature-flag-lint.yml`: grep removed flags(ADR-046)

任一失败 → 阻塞 merge。配 GitHub branch protection 强制。

§9 前端约定

9.1 app router 目录约定

- `(public)/` 路由组: 无需登录、支持 ISR, 例 `/u/[username]`、`/w/[work_id]/[slug]`

- `(private)/` 路由组: `middleware.ts` 强制 redirect 未登录到 `/login`, 例 `/library`、`/dashboard`
- `(auth)/`: 登录 / 注册 / 注销页本身
- `api/`: 仅 webhook / cron; 业务调用统一走 **Server Action**, 不写自定义 REST 端点

9.2 Server Component vs Client Component

- 默认 Server Component。需 hooks / browser API 才加 `"use client"`
- **禁止**在 Client Component 直接 import `lib/supabase/server.ts` 或 `admin.ts`
- 表单提交: `<form action={serverAction}>` + `useFormState` / `useFormStatus`, 不写 `fetch`

9.3 表单 stack

- 校验: `zod` (与 server action 共享 schema 文件)
- 表单状态: `react-hook-form` + `@hookform/resolvers/zod`
- UI: shadcn `<Form>` + `<FormField>` (底层 react-hook-form)
- **唯一约定**: 表单 schema 同时被 client RHF 与 server action 引用; 两边一份 zod schema 文件

9.4 Client state 选型

按优先级:

1. **URL state**(`useSearchParams` / `pathname`) — 首选, SSR 友好 + 分享/刷新保留
2. **Server state**(RSC / Server Action `revalidatePath`) — 数据类
3. `useState` (组件本地) — 临时 UI 状态
4. **zustand**(跨组件共享但非 server) — **仅在 1-3 都不适合时才用**; P0 暂不引入, 触发场景必走 ADR

9.5 Loading / Error / NotFound 约定

每个路由段(尤其 `(private)/library`、`(public)/u/[username]`) 必须有:

- `loading.tsx` Skeleton(不超过 3 个 placeholder 形状)
- `error.tsx` 用 `ErrorCode` 渲染 (§6.2a), 带"重试"按钮触发 `reset()`
- `not-found.tsx` 友好 404, 带返回入口

9.6 组件分层

- `components/ui/`: shadcn 原子(`Button`、`Input`、`Form`),零业务逻辑
- `components/domain/`: 业务复合组件(`EntryCard`、`ProgressLogger`),依赖 ui,可调 server action
- 业务组件不直接调 supabase,只通过 **server action**

9.7 样式

- Tailwind v3 + shadcn 默认主题。颜色用 CSS 变量(`hsl(var(--primary))`),P1 接 theme switch 无返工
- 自定义颜色 / 字体一律走 `tailwind.config.ts`,不内联

9.8 国际化(PO 字符串规范)

- 任何 UI 文案必须走 `t("...")`,不在 JSX 拼接字符串(连"加载中..."都不能硬编码)
- 文案 key 命名: `<domain>.<element>.<state>`,例 `library.empty.title`
- 详见  [i18n key bundle 雏形](#)

§10 Schema 演进:expand-migrate-contract 范例

场景:P1 要在 `entries` 表加 `quality_state TEXT NULL` (质量状态:awesome / fine / regret)。按以下三阶段走:

阶段 A:expand(扩张兼容)

新 migration `YYYYMMDDHHMM_add_entries_quality_state.sql`:

```
-- 加可空字段,旧代码不感知
ALTER TABLE entries ADD COLUMN quality_state TEXT;

-- 加 CHECK 但允许 NULL
ALTER TABLE entries ADD CONSTRAINT entries_quality_state_ch
k
CHECK (quality_state IS NULL OR quality_state IN ('awesom
e','fine','regret'));

-- pgTAP 测试:旧 entry NULL 不影响 RLS
```

部署后:

- 旧代码继续工作(字段对它不存在)

- 新代码可读可写,后端 feature flag `quality_state_enabled` 控(ADR-046)

阶段 B:migrate(数据迁移 + 双写)

可选回填:

```
-- 例:把 finished + rating >= 4 标为 'fine'
UPDATE entries SET quality_state = 'fine'
WHERE status = 'finished' AND rating >= 4.0 AND quality_state IS NULL;
```

应用层:

- UI 添加质量状态选择器
- Server Action 双写新字段
- 旧客户端继续不看此字段

阶段 C:contract(收紧)

90d 后所有客户端都升级,新 migration:

```
-- 若业务要求必填
ALTER TABLE entries ALTER COLUMN quality_state SET DEFAULT 'fine';
UPDATE entries SET quality_state = 'fine' WHERE quality_state IS NULL;
ALTER TABLE entries ALTER COLUMN quality_state SET NOT NULL;

-- 删除旧 feature flag 代码路径
```

并删除 ADR-046 90d 中失效 flag(走 `feature-flag-lint.yml` 强制)。

红线

- **✗ 永不在一个 migration 里同时 ADD + NOT NULL + 无 DEFAULT:**会锁表 + 旧代码 INSERT 立崩
- **✗ 永不直接 DROP COLUMN:**expand 阶段先标记 `deprecated`,90d 后再 drop
- **✗ 改 fingerprint normalize 规则等同改 schema(见 ADR-049)**

- **✗ 永不跳过 expand 直接 contract**(生产事故主要源)

§11 本页与其它页的关系

- 本页 = **代码与运行时模板**(文件布局 / SQL / TS / pgTAP / 前端约定)
- DB Schema = **运行时定顿**(表 / 字段 / RLS 矩阵) →  DB Schema v1.0.0 设计
- ADR 志 = **决策理由** → 在
- v6 = **产品不变量** →  个人书影剧库平台产品方案 v6

任一冲突:以 v6 为准,依次向下。